

8byte DevOps Assignment

Project Analysis & Technical Documentation

Repository: github.com/arshitchoubey18/8byte-devops-assignment

Author: Arshit Choubey

Live Demo: arshit-8byte-alb-595835421.us-east-1.elb.amazonaws.com (AWS Academy lab)

Executive Summary

This project implements a production-grade, containerized web application on AWS using modern DevOps practices. It demonstrates end-to-end Infrastructure as Code (IaC), automated CI/CD, observability, and security hardening — all built around AWS serverless containers (ECS Fargate).

The solution follows a classic 3-tier pattern: Application Load Balancer (public), ECS Fargate tasks (application), and RDS PostgreSQL (private data layer), provisioned entirely with Terraform.

1. Architecture Overview

The architecture is designed for high availability, security, and cost efficiency across two Availability Zones in us-east-1.

Layer	Implementation
Compute	ECS Fargate (serverless containers)
Network	VPC 10.0.0.0/16 with 2 public + 2 private subnets across us-east-1a/b
Database	RDS PostgreSQL 15 in private subnets
Load Balancer	Application Load Balancer (ALB)
Container Registry	ECR
IaC & State	Terraform with S3 backend + DynamoDB locking

Key Design Principles

- Least-privilege security groups isolate ALB → App → DB traffic
- Private subnets for data tier, no direct internet access for RDS
- Stateless containers enable horizontal scaling via Fargate
- Immutable infrastructure through Terraform

2. Part 1 – Infrastructure Provisioning (Terraform)

All infrastructure is codified under the /terraform directory and provisioned with a single apply command:

```
cd terraform
terraform init
terraform apply -var="db_password=YourSecurePass123"
```

What gets created:

- VPC with public/private subnets, route tables, Internet Gateway, NAT Gateway
- Security Groups: ALB (80/443), ECS (from ALB only), RDS (5432 from ECS only)
- ECS Cluster + Fargate Service + Task Definition
- ALB with target group and health checks
- RDS PostgreSQL 15 (db.t3.micro) with automated backups
- ECR repository for application images
- S3 bucket for Terraform state with versioning
- DynamoDB table for state locking

3. Part 2 – Deployment Automation (CI/CD)

GitHub Actions pipeline defined in .github/workflows/ci-cd.yml implements a trunk-based workflow:

Trigger	Actions	Outcome
Pull Request	pytest unit tests	Quality gate before merge
Merge to main	Build Docker → Trivy scan → Push to ECR	Secure image in registry
Staging deploy	Auto-update ECS service	Immediate feedback
Production deploy	Manual approval → ECS update	Controlled release

Security scanning with Trivy ensures vulnerabilities are caught before deployment, aligning with DevSecOps best practices.

4. Part 3 – Monitoring and Logging

Observability is implemented via native AWS CloudWatch:

Dashboard: monitoring/cloudwatch-dashboard.json – pre-built dashboard

Metrics tracked: ECS CPU/Memory Utilization, ALB RequestCount & TargetResponseTime, RDS DatabaseConnections

Logs: All container stdout/stderr streamed to /ecs/8byte-app log group

Alarms (recommended): High CPU >80%, ALB 5xx >5, RDS connections >80%

5. Part 4 – Best Practices Implemented

Security

- RDS deployed in private subnets only – no public endpoint
- AWS SSM Parameter Store (SecureString) for secrets, not hardcoded
- Security Group isolation: ALB → ECS → RDS
- IAM roles with least-privilege for ECS task execution
- Container image scanning via Trivy in CI pipeline

Cost Optimization

- Fargate serverless – pay per vCPU/memory, no idle EC2 instances
- db.t3.micro for development workloads
- 7-day automated RDS backups (vs 35-day default)
- Single NAT Gateway for dev environment (not per-AZ)

Secret Management & Backup

- Secrets: AWS SSM Parameter Store with encryption at rest
- Backups: RDS automated snapshots (7 days) + S3 versioned Terraform state for disaster recovery

6. Challenges Faced & Resolutions

The repository includes docs/CHALLENGES.md documenting five real-world issues encountered during implementation (per README). Based on typical patterns for this stack, these likely include:

VPC/Subnet routing: Private tasks needing NAT for ECR pull – resolved by adding NAT Gateway and proper route tables

RDS connectivity: Security group misconfiguration blocking ECS → RDS on 5432 – fixed with least-privilege SG rules

Terraform state locking: Concurrent applies causing corruption – implemented DynamoDB locking

ALB health checks failing: Container port mismatch or slow startup – tuned health check grace period

CI/CD permissions: GitHub Actions lacking ECR/ECS IAM rights – created dedicated OIDC role with scoped policies

7. How to Reproduce

1. Clone repo and configure AWS credentials (AWS Academy or personal account)
2. Create S3 bucket and DynamoDB table for Terraform backend
3. Run: `terraform init && terraform apply -var="db_password=..."`
4. Push code – GitHub Actions builds image, scans, pushes to ECR
5. ECS service pulls new image and rolls out via ALB
6. Access application via ALB DNS name
7. View metrics in CloudWatch dashboard

8. Assessment

This assignment successfully demonstrates a complete DevOps lifecycle: IaC provisioning, containerization, secure CI/CD, and observability on AWS. The design choices – Fargate over EC2, private RDS, SSM for secrets, Trivy scanning, and manual production approval – reflect production readiness rather than a toy demo. It is well-suited as a template for startups needing a low-ops, scalable foundation.

Appendix: Repository Structure (inferred)

```
8byte-devops-assignment/  
├─ terraform/          # VPC, ECS, ALB, RDS, ECR, IAM  
│   ├── main.tf  
│   ├── variables.tf  
│   └─ outputs.tf  
├─ .github/workflows/  
│   └─ ci-cd.yml        # PR tests, build, scan, deploy  
├─ app/                 # Application source (Flask/Django/Node)  
│   ├── Dockerfile  
│   └─ tests/  
├─ monitoring/  
│   └─ cloudwatch-dashboard.json  
└─ docs/  
    └─ CHALLENGES.md
```
